AV03 — Projeto Aplicado I

Schulz S.A. — Construção do MVP (Parte 1)

Unidade curricular: Projeto Aplicado I

Empresa parceira: Schulz S.A.

Professores: Hugo Menezes Barra e Rodrigo Losina

Linguagem: Python 3

Banco de dados: SQLite

Data: 03/11/2026

1. Objetivo

Desenvolver a primeira parte do MVP de um sistema de controle

de calibrações, utilizando Programação Orientada a Objetos

(POO) e integração com banco de dados relacional SQLite.

O sistema permite cadastrar profissionais, equipamentos

e medições, verificar tolerâncias e gerar relatórios.

2. Estrutura do projeto

- 04_classes.py: classes do sistema e operações SQLite.
- 05_main.py: execução e demonstração do MVP.
- 06_mvp_parte1.md: documentação técnica.
- 03_banco.sql: script SQL da etapa anterior (AV02).

O banco SQLite utilizado nos testes é uma cópia local

da base desenvolvida na AV02.

3. Programação Orientada a Objetos

O sistema foi organizado em quatro classes:

Tecnico

Representa os profissionais envolvidos nas calibrações.

Atributos:

- nome
- tipo

O tipo pode ser TECNICO ou SOLICITANTE.

Equipamento

Representa os instrumentos utilizados nas medições.

Atributos:

- codigo
- denominacao
- fabricante

O código identifica cada equipamento de maneira única.

Medicao

Representa os resultados obtidos durante a calibração.

Atributos principais:

- id_equipamento
- id_tecnico
- data_medicao
- parametro
- valor_nominal
- tolerancia_positiva
- tolerancia_negativa
- valor_obtido

Métodos:

- calcular_erro()
- validar_status()

BancoDados

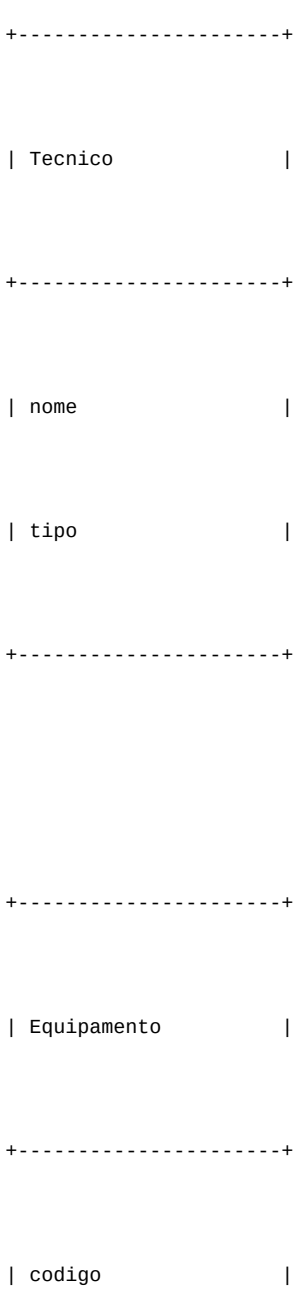
Responsável pela conexão com o SQLite e pelas operações

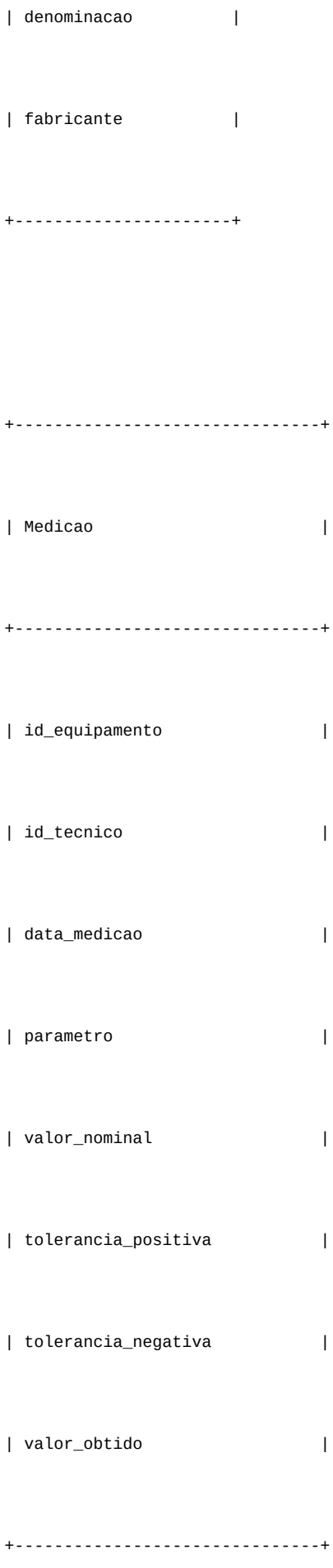
de persistência e consulta.

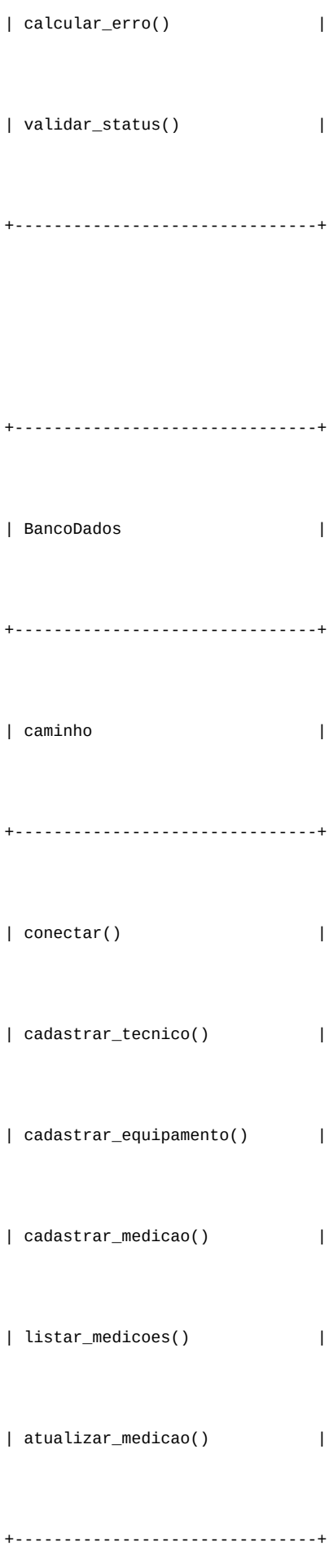
Métodos:

- conectar()
- cadastrar_tecnico()
- cadastrar_equipamento()
- cadastrar_medicao()
- listar_medicoes()
- atualizar_medicao()

4. Diagrama textual das classes




| denominacao |
| fabricante |
+----------+
+----------+
| Medicao |
+----------+
| id_equipamento |
| id_tecnico |
| data_medicao |
| parametro |
| valor_nominal |
| tolerancia_positiva |
| tolerancia_negativa |
| valor_obtido |
+----------+


| calcular_erro() |
| validar_status() |
+------------------------------+
+------------------------------+
| BancoDados |
+------------------------------+
| caminho |
+------------------------------+
| conectar() |
| cadastrar_tecnico() |
| cadastrar_equipamento() |
| cadastrar_medicao() |
| listar_medicoes() |
| atualizar_medicao() |
+------------------------------+

5. Regras de negócio

O erro é calculado pela diferença entre o valor obtido

e o valor nominal:

Erro = Valor obtido - Valor nominal

O intervalo permitido é:

Limite inferior = Valor nominal - Tolerância negativa

Limite superior = Valor nominal + Tolerância positiva

Quando o valor obtido estiver dentro desse intervalo,

incluindo os limites, a medição recebe status CONFORME.

Caso contrário, recebe status NAO CONFORME.

No roteiro da AV03, esses resultados correspondem

respectivamente a APROVADO e REPROVADO.

Os valores CONFORME e NAO CONFORME foram mantidos

para respeitar a estrutura existente no banco da AV02.

6. Integração com SQLite

A conexão utiliza a biblioteca sqlite3 do Python.

Foram implementadas as operações:

- INSERT para cadastrar profissionais, equipamentos e medições.
- SELECT com JOIN para gerar o relatório de calibração.
- UPDATE para atualizar resultados de medições.

As consultas utilizam parâmetros SQL para evitar

a concatenação direta de valores nos comandos.

As chaves estrangeiras são habilitadas pela instrução:

PRAGMA foreign_keys = ON

7. Tratamento de erros

O sistema utiliza ValueError para validar informações

de entrada, como nomes vazios, tipos inválidos

e tolerâncias negativas.

O programa principal captura erros SQLite e ValueError.

As conexões são fechadas por blocos finally.

Em caso de erro durante uma operação SQL, não ocorre

o commit dessa operação.

8. Testes realizados

Teste 1 — Medição conforme

Valor nominal: 100

Tolerâncias: +5 e -5

Valor obtido: 103

Erro: 3

Resultado: CONFORME

Teste 2 — Medição não conforme

Valor nominal: 100

Tolerâncias: +5 e -5

Valor obtido: 108

Erro: 8

Resultado: NAO CONFORME

Teste 3 — Consulta relacional

A execução do programa principal retornou três

medições utilizando JOIN entre as tabelas

medicoes, equipamentos e tecnicos.

Teste 4 — Atualização

A medição de demonstração de ID 3 foi atualizada.

Novo valor obtido: 104

Novo erro: 4

Novo status: CONFORME

Quantidade de registros atualizados: 1

9. Segurança

O projeto não necessita de senhas para conexão

com o banco SQLite local.

Os comandos SQL utilizam parâmetros.

O arquivo .gitignore impede o envio automático

de bancos locais e arquivos de credenciais

que ainda não estejam versionados.

Antes da publicação, é necessário verificar

também os arquivos já rastreados e o histórico Git.

10. Como executar

Abrir o terminal na pasta entrega-03.

Verificar a sintaxe:

python3 -m py_compile 04_classes.py 05_main.py

Executar a demonstração:

python3 05_main.py

O programa demonstra o cadastro, a validação

e a geração do relatório no terminal.

Observação: o programa gera um identificador

único para cada equipamento de demonstração,

permitindo novas execuções sem conflito de código.

Cada execução acrescenta novos registros de

teste ao banco SQLite local.

11. Conclusão

O MVP Parte 1 demonstrou a utilização de classes

Python, regras de validação de tolerâncias e

persistência em SQLite.

Foram executados testes de cadastro, consulta

relacional, cálculo de erro, classificação

de medições e atualização de registros.

O sistema constitui uma base para as próximas

etapas de desenvolvimento do projeto.

12. Inicializacao automatica do banco

Quando o arquivo schulz_calibracao.db nao existe,

o programa principal utiliza o script 03_banco.sql

para criar automaticamente as tabelas SQLite.

Essa funcionalidade foi testada em uma pasta separada,

simulando a primeira execucao no computador do professor.

O teste confirmou a criacao do banco, o cadastro

de duas medicoes e a geracao do relatorio de calibracao.

Para reproduzir o teste, basta executar:

python3 05_main.py

O arquivo 03_banco.sql deve estar na mesma pasta

dos arquivos Python.

Cada nova execucao adiciona registros de demonstracao

ao banco local.